Analysis of implementation weight distribution comparison into network training process.

Deliverable D10 · Work Package WP3

Summary - Report

Project identification:
Project title: A Statistical Approach to Monitor Quantisation in Neural Network Training
Project acronym: SAMQ-NN
Project code: 09I03-03-V04-00562
Programme: Recovery and Resilience Plan of the Slovak Republic
Component: 9. More effective governance and strengthening of the funding of research, development and innovation under the Recovery and Resilience Plan of the Slovak Republic
Investment: 3. Excellent science
Aid scheme: State aid scheme to support research, development and innovation under Component 9 of the Recovery and Resilience Plan of the Slovak Republic, No. SA.106633
Project start: 06/2024
End of implementation: 05/2026
Principal investigator: Ing. Roman Budjač, PhD.

1. Introduction

The present document is the deliverable D10 of work package WP3 (Development and implementation of a new training strategy) of the SAMQ-NN project. The analysis directly follows on the results of the previous packages: in WP1, suitable statistical methods for comparing the weight distribution of a neural network were selected and tested, such as Kullback-Leibler divergence (KL), Jensen-Shannon divergence (JS) and cosine similarity. In the case of WP2, these methods were implemented into the model quantisation process.

While in the previous packages the comparison of the weight distribution was performed mostly statically, that is, only after the completion of quantisation over the stored weights of the original and the quantised model, the focus of WP3 is to move this comparison directly into the training process. The main methodological idea is to monitor the similarity index between the reference weight distribution of the original (non-quantised) model and the current weight distribution of the quantised model.

The aim of this analysis is to examine in what way it is possible to integrate such a comparison into the training loop, which of the available statistical methods are the most suitable for continuous monitoring, what the computational consequences of such integration are, and how the obtained similarity index can be used to control the training itself. The conclusions of the analysis form a direct input for the practical implementation in the Deliverable D12.

2. Baseline architecture and statistical methods

The analysis is based on the statistical core implemented in WP2 (deliverable D7). This provides the computation of KL divergence, JS divergence and cosine similarity, and additionally also the Wasserstein distance. The common basis of these metrics is the representation of a set of weights as a discrete probability distribution obtained by a histogram.

2.1 Representation of the weight distribution

The weights of the selected layer (or of the entire model) are summarised into a histogram over fixed bin boundaries (intervals).The weights of the selected layer (or of the entire model) are summarised into a histogram over fixed bin boundaries (intervals). In order for two distributions to be comparable, identical bin boundaries are used for both the reference and the current distribution. The histogram is then normalised into a probability distribution (the sum of the values equal to one). Before normalisation, a small constant epsilon (e.g. 1e-10) is added to the counts, which ensures numerical stability and prevents the computation of the logarithm of zero in KL and JS divergence.

2.2 Properties of the methods from the point of view of monitoring during training

The following mathematical properties are essential when selecting a method for continuous monitoring:

- KL divergence: asymmetric and unbounded. Intuitively it expresses the degree of deviation of the quantised distribution from the reference one. Even though it is not bounded, from the magnitude of the value it is possible to estimate fairly directly the extent of the difference between the distributions, which is why it is suitable as the primary index.

- JS divergence: symmetric and bounded in the interval [0, 1]. It provides a stable output suitable for thresholding and optimisation criteria, and it is robust even in the case of non-overlapping supports, where KL divergence would lead to an undefined result.

- Cosine similarity: measures the orientation of the weight vectors independently of their magnitude, thereby complementing the previous metrics with a structural view of the change in the weights.

From the monitoring point of view, it is advantageous that these methods are not computationally demanding, so that in the future it is also possible to compute several indices simultaneously and compare their evolution.

3. Proposal for integration into the training process

3.1 Reference

Before launching the fine-tuning of the quantised model, the weights are extracted from the original model and their distribution is stored as a reference. This reference distribution does not change throughout the entire fine-tuning and serves as a fixed point of comparison. At the same time, the bin boundaries are also fixed, so that every subsequent comparison uses a consistent metric.

3.2 Computatuon mechanism

The continuous comparison is implemented by means of a custom class of the callback type within the Keras framework. Callbacks are a set of methods called at various phases of training, testing and prediction, and they are suitable for gaining insight into the internal states and statistics of the model during training. Specifically, the method called at the end of each epoch is used: the current weights of the quantised model are extracted, their histogram is built over the same bin boundaries as the reference, the similarity index is computed by the chosen method, and the value is written into the training history. The structure of such a callback is illustrated by the following code:

```
class SimilarityMonitorCallback(keras.callbacks.Callback):
    def __init__(self, reference_dist, bin_edges, method="kl"):
        super().__init__()
        self.reference_dist = reference_dist   # rozdelenie vah povodneho modelu
        self.bin_edges = bin_edges             # pevne hranice binov
        self.method = method                   # "kl" | "js" | "cosine"
        self.history = []

    def on_epoch_end(self, epoch, logs=None):
        current = flatten_weights(self.model.get_weights())
        index = compare_distributions(self.reference_dist, current,
                                      self.bin_edges, self.method)
        self.history.append(index)
        logs[f"sim_{self.method}"] = index
        print(f"Epoch {epoch+1}: index podobnosti "
              f"({self.method}) = {index:.4f}")
```

The functions flatten_weights and compare_distributions represent helper routines that will be implemented in full form in later deliverables.

3.3 Comparison by layers versus globally

The analysis brings to our attention two views. The comparison by individual layers provides finer information about which layers are most affected by quantisation, and it is useful in the design of adaptive quantisation strategies with a different number of bits for different layers. The global index aggregates all the weights into a single value and provides a summary indicator of the state of the model during training. For the purposes of continuous monitoring, it is recommended to monitor the global index as the main metric and to analyse by layers in the case of detection of significant divergence.

4. Computational demands and impact on training

The computation of the index consists of building the histogram and evaluating the entropy, or the dot product in the case of cosine similarity. The overhead of these operations is by orders of magnitude negligible in comparison with a single forward and backward pass of the network. From the observations in WP2 it followed that computing the index after each epoch did not have a significant effect on the course of training, not even with the simultaneous computation of several indices. The integration of monitoring into training is therefore practically free from the performance point of view, which is an important prerequisite for its deployment in a real training cycle.

5. Use of the index during training

The availablity of similarity index opens up several possibilities for controlling the training that are not available in static (post-quantisation) comparison:

- Early termination of training: if the accuracy of the quantised model does not increase during training and the index has stabilised without further improvement, it is possible to terminate the training earlier and save computational time.

- Adjustment of quantisation parameters: if the difference between the distributions is significant, it is possible to change the number of quantisation bits and repeat the process with a more favourable setting.

- Interpretation of degradation: a significantly increasing or high divergence signals degradation of the weights due to quantisation, that is, an undesirable impact of quantisation on the network.

The index thereby provides information about the difference of the weight distributions, not directly about the accuracy of the model. Therefore it is recommended to monitor the index in combination with the accuracy on the test set, which provides a more complete picture of the relationship between the change in the weight distribution and the performance of the model.

6. Validation of implementation into training criteria (WP3.2)

In accordance with the aim of WP3.2 (implementation of statistical methods into the training process and validation against predetermined criteria), the analysis proposes the following validation criteria for verifying the integrated solution:

- Threshold value of the index: setting a threshold (especially for the bounded JS divergence) above which the distribution is considered significantly different.

- Correlation of the index with accuracy: verifying whether the evolution of the similarity index is meaningfully related to the evolution of the accuracy of the quantised model on the test set.

- Stability across epochs: the index should evolve smoothly and reproducibly during training, without unjustified jumps.

- Reproducibility of implementaition across datasets and architectures: verifying consistent behaviour on the MNIST (or modifications), CIFAR datasets, with a view to a more extensive dataset (ImageNet) or its subset.

7. Inputs for the implementation

The following specific design decisions follow from the analysis and will be reflected in the implementation:

- The use of uniform quantisation with a selectable number of bits; the creation and storage of the reference weight distribution before fine-tuning;
- fixing the bin boundaries for consistent comparison; the implementation of the display by means of a Keras callback type class with the computation of the index at the end of each epoch;
- support for the KL, JS and cosine similarity methods;
- a dataset-agnostic design with support for MNIST, CIFAR with the possibility of extension to ImageNet (or a subset);
- and the output of the similarity index and accuracy during training (with the descriptions of the graphs in the English language), so that the user can monitor its value.

The presented procedure may be modified with respect to the circumstances.

8. Conclusion

The analysis demonstrated that the comparison of the weight distribution can be integrated directly into the training process of the quantised model by means of the callback mechanism, with practically negligible computational overhead. The similarity index obtained in this way provides the developer with continuous feedback on the impact of quantisation on the model's weights and enables informed decisions during training. From early termination to the adjustment of quantisation parameters. The proposed validation criteria form a framework (concept, framework) for verifying the solution. This analysis is the basis for the implementation of the new training strategy in deliverable D12 and further deliverables.